

Unidad Académica de Economía

SEMANA 9

Por: Gil Alexander Ramirez

Carrera: Sistemas computacionales

Materia: Estructura de datos básica

Docente: Eligardo Cruz Sanchez

Reto IA - Comprende: Trazado Socrático de Merge Sort

Estado inicial: [7, 2, 9, 4, 1, 8, 3, 5]

Primer paso (primera división): [7, 2, 9, 4] | [1, 8, 3, 5]

Cada mitad se vuelve a dividir otra vez: [7, 2] [9, 4] | [1, 8] [3, 5]

PREGUNTA CLAVE (merge)

¿Qué propiedad garantiza que merge() siempre esté ordenado?

La clave es que merge siempre junta dos subarreglos que ya están ordenados de antes. Va comparando los primeros elementos de cada lado y pone el más chico primero, manteniendo todo en orden en cada paso.

RESULTADO FINAL DEL TRAZADO

Después de dividir y fusionar: [1, 2, 3, 4, 5, 7, 8, 9]

IA-REFLEXION-C:

La IA explico el merge() como si fueran dos filas ordenadas donde solo miras al de adelante. Tomas el menor de ambos lados y así nunca rompes el orden. Esa idea me dejó más claro porque el resultado final queda ordenado siempre.

Checkpoint de Comprensión

1. ¿Por qué Merge Sort garantiza $O(n \log n)$ incluso en el peor caso?

Porque siempre parte el arreglo exactamente a la mitad, sin importar como vengan los datos acomodados.

Propiedad clave del árbol:

- Tiene altura $\log_2(n)$ por dividirse entre 2 cada vez.
- En cada nivel se procesan n elementos en el merge.
- **Trabajo total = $n \times \log n = O(n \log n)$.** Y esto no cambia para nada (sea el mejor, promedio o el peor caso).

2. Si el árbol tiene 20 niveles, ¿cuántos elementos hay?

Si niveles = $\log_2(n)$, entonces $n = 2^{20} = 1,048,576$ elementos en el arreglo.

3. ¿Qué pasa si usamos < en vez de <=?

Se pierde la estabilidad. Osea, si hay elementos iguales, podrian cambiar de posicion entre ellos.

Ejemplo concreto:

Arreglo original ordenado por fecha: [("A", 90), ("B", 90)]

- Si usas <=, el resultado queda: [("A", 90), ("B", 90)] (se mantiene).
- Si usas <, el resultado queda: [("B", 90), ("A", 90)] (se invierte el orden porque toma primero el de la derecha).

4. ¿Por qué la estabilidad importa en StreamMX?

Porque los datos ya venian ordenados por fecha. Al ser Merge Sort estable, si dos pelis tienen el mismo puntaje, NO cambia su orden original. Las pelis con igual puntaje mantienen su orden cronologico.

Implementación, Estabilidad y Benchmark vs S8

Reto 1

```
#!/usr/bin/env python3
import time
import random
import copy

def merge_sort(arr, inicio, fin):
    if inicio >= fin:
        return
    medio = (inicio + fin) // 2
    merge_sort(arr, inicio, medio)
    merge_sort(arr, medio + 1, fin)
    merge(arr, inicio, medio, fin)

def merge(arr, inicio, medio, fin):
    izquierda = arr[inicio:medio + 1]
    derecha = arr[medio + 1:fin + 1]
    i = j = 0
```

```

k = inicio
while i < len(izquierda) and j < len(derecha):
    if izquierda[i] <= derecha[j]:
        arr[k] = izquierda[i]
        i += 1
    else:
        arr[k] = derecha[j]
        j += 1
    k += 1
while i < len(izquierda):
    arr[k] = izquierda[i]
    i += 1
    k += 1
while j < len(derecha):
    arr[k] = derecha[j]
    j += 1
    k += 1

```

ESTABILIDAD

```

def merge_sort_estable_demo():
    registros = [
        ("Pelicula_C", 90, 1),
        ("Pelicula_A", 85, 2),
        ("Pelicula_E", 90, 3),
        ("Pelicula_B", 75, 4),
        ("Pelicula_D", 85, 5)
    ]
    registros.sort(key=lambda x: x[1])
    print("Ordenados por puntaje:")
    for r in registros:
        print(r)

```

#REFLEXIONA-2:

Si se mantuvo el orden. Las películas con el mismo puntaje conservaron su orden cronológico. Un algoritmo inestable cambiaría eso.

BUBBLE SORT

```
def bubble_sort_s8(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

MEDICIÓN Y BENCHMARK

```
def medir_tiempo(func, arr, *args):
    copia = copy.deepcopy(arr)
    inicio = time.perf_counter()
    func(copia, *args)
    fin = time.perf_counter()
    return round(fin - inicio, 6)

def generar_arreglo(n, tipo):
    if tipo == "aleatorio":
        return [random.randint(0, n) for _ in range(n)]
    elif tipo == "ordenado":
        return list(range(n))
    elif tipo == "inverso":
        return list(range(n, 0, -1))
```

VALIDA:

Merge Sort crece mucho mas lento que Bubble Sort. Al aumentar n, Bubble crece de forma casi cuadratica, mientras que Merge crece de forma logaritmica.

MAIN

```
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]
    merge_sort(arr, 0, len(arr) - 1)
    print("Ordenado:", arr)
    print("\n--- Estabilidad ---")
    merge_sort_estable_demo()
    print("\n--- Benchmark ---")
    benchmark_completo()
```

IA-REFLEXION-A:

La pregunta de que pasaba si escribías directo sobre arr nos hizo ver que perderíamos datos antes de compararlos. Ahí entendimos porque los arreglos auxiliares son obligatorios.

Reto 2 — Estabilidad en Datos Reales: el Caso StreamMX

```
def experimento_estabilidad_streamMX():
    registros = [
        ("A", 90, 1),
        ("B", 85, 2),
        ("C", 90, 3),
        ("D", 85, 4),
        ("E", 90, 5)
    ]
    originales_90 = [r for r in registros if r[1] == 90]

def merge_sort_tuplas(arr, inicio, fin):
    if inicio >= fin:
        return
    medio = (inicio + fin) // 2
    merge_sort_tuplas(arr, inicio, medio)
    merge_sort_tuplas(arr, medio + 1, fin)
    merge_tuplas(arr, inicio, medio, fin)

def merge_tuplas(arr, inicio, medio, fin):
    izquierda = arr[inicio:medio + 1]
    derecha = arr[medio + 1:fin + 1]
    i = j = 0
    k = inicio
    while i < len(izquierda) and j < len(derecha):
        if izquierda[i][1] >= derecha[j][1]:
            arr[k] = izquierda[i]
            i += 1
        else:
            arr[k] = derecha[j]
            j += 1
        k += 1
```

```

while i < len(izquierda):
    arr[k] = izquierda[i]
    i += 1
    k += 1
while j < len(derecha):
    arr[k] = derecha[j]
    j += 1
    k += 1

copia = registros[:]
merge_sort_tuplas(copia, 0, len(copia) - 1)
resultado_90 = [r for r in copia if r[1] == 90]
originales_titulos = [r[0] for r in originales_90]
resultado_titulos = [r[0] for r in resultado_90]

assert originales_titulos == resultado_titulos, "El algoritmo NO es estable"
print("Merge Sort es estable: se preservó el orden cronológico")

if __name__ == "__main__":
    experimento_estabilidad_streamMX()

```

REFLEXIONA-2:

Un algoritmo inestable cambiaría el orden de películas con igual puntaje, mostrando contenido nuevo antes del viejo sin ningún sentido. Esto le pega feo a la experiencia de usuario.

IA-REFLEXION-B:

La pregunta de 'si dos películas tienen el mismo puntaje, cuál debería ir primero' nos enseña que el orden original importa mucho. La estabilidad afecta directamente la experiencia del usuario.

Reto IA — Reflexiona: Simulacro de Defensa Oral (Merge Sort)

Exposición inicial:

Merge Sort asegura que `merge()` saca un subarreglo ordenado porque combina dos subarreglos que ya están ordenados, eligiendo siempre el menor (o mayor). Su complejidad es $O(n \log n)$ porque el arreglo se divide a la mitad ($\log n$ niveles) y en

cada nivel se procesan n elementos. En el benchmark se ve que Merge es mucho mas eficiente que Bubble en arreglos grandes.

Pregunta técnica 1: ¿Por qué merge() nunca rompe el orden si los subarreglos ya están ordenados?

Respuesta: Porque en cada paso solo comparas el primer elemento de cada subarreglo y eliges el menor. Como ya venian ordenados de antes, nunca metes algo fuera de lugar.

Pregunta técnica 2: ¿Qué pasaría si los subarreglos no estuvieran ordenados antes de hacer merge()?

Respuesta: El resultado final no quedaria ordenado porque merge() asume como condicion que cada subarreglo ya esta ordenado. Si eso falla, el algoritmo truena.

VEREDICTO FINAL: Aprobado

REFLEXIONA-1: El equipo acerto en que Merge Sort seria mas rapido. En n grandes Bubble es inviable y Merge sigue siendo eficiente.

REFLEXIONA-2: La estabilidad hace que el ranking sea consistente y no confunda al usuario.

REFLEXIONA-3: Merge Sort gasta memoria extra por los arreglos auxiliares, lo cual es un problema con sistemas limitados. El reto que sigue es hacerlo in-place.

IA-REFLEXION-R: La pregunta que costo fue que pasaba si los subarreglos no estaban ordenados. Ahi entendimos que es una precondition super critica.

Reto IA — Valida: Diseño de Casos Adversariales para Merge Sort

VALIDA: caso adversarial 1 $\rightarrow [1, 2]$

VALIDA: caso adversarial 2 $\rightarrow [3, 5, 8]$

VALIDA: caso adversarial 3 $\rightarrow [(85, 'B'), (90, 'A'), (90, 'C')]$

VALIDA: caso adversarial 4 $\rightarrow [2, 3, 7, 8, 10]$

IA-REFLEXION-V:

El caso 4 nos mostro un bug donde los subarreglos no se copiaban bien cuando inicio no era 0 por culpa de los indices del slicing. Se corrigio ajustando los rangos a inicio:medio+1 y medio+1:fin+1.

Reto IA — Profundiza: Diagnóstico de la Restricción de Memory

Para StreamMX el algoritmo ideal debe mantener $O(n \log n)$ pero bajar la memoria auxiliar a $O(\log n)$ u $O(1)$, trabajando directo en el arreglo original sin tanta copia.

Para $n = 4,000,000$ elementos:

- Merge Sort normal usa $O(n)$ espacio $\rightarrow \sim 4,000,000$ posiciones extra.
- Si usamos $O(\log n)$ $\rightarrow \log_2(4,000,000) \approx 22$ posiciones.
- El ahorro es brutal: pasamos de millones a solo decenas en memoria, quitando el riesgo de que colapse la RAM.

*IA-REFLEXION-P*:

La pregunta clave de cuándo los subarreglos son más grandes nos hizo ver que pasa al mero final, en la fusión mas alta. Ahi es cuando el consumo de RAM llega a su punto maximo.